CASE STUDY 3: BANK ACCOUNT AND MONEY TRANSFERS

Q1. Identification of Domain Invariants (6 Marks)

An invariant is a condition or business rule that must always remain true for an object to be considered valid.

In a banking application, invariants are important because they prevent invalid transactions and maintain the correctness of account information.

The following are the important business invariants:

1. Account Balance Must Respect the Overdraft Limit

The customer should not be allowed to withdraw more money than the available balance and permitted overdraft limit. This rule is important because it prevents customers from using more money than the bank allows. If this rule is violated, the bank may face financial losses and incorrect account balances.

2. Transaction Amount Must Always Be Positive

Every deposit, withdrawal and money transfer must have an amount greater than zero. Zero or negative amounts should not be accepted. This rule prevents invalid transactions and avoids incorrect balance calculations.

3. Frozen Accounts Must Not Allow Withdrawals

When an account is frozen, the customer should not be allowed to withdraw money or transfer funds from that account. This rule is important for preventing unauthorized transactions and protecting customer money. If violated, fraudulent transactions may take place.

4. Closed Accounts Must Not Allow Transactions

Once an account is closed, it should not accept further deposits, withdrawals or fund transfers. This rule ensures that closed accounts cannot be used again. Otherwise, incorrect financial records and accounting problems may occur.

5. Money Transfer Must Be Completed Successfully

When money is transferred from one account to another, the amount must be deducted from the sender's account and credited to the receiver's account. Both operations should succeed together. If only one operation succeeds, money may be lost or account balances may become inconsistent.

6. Account Balance Must Match Transaction History

The account balance must always match the deposits, withdrawals and transfers recorded in the transaction history. This rule is important for maintaining accurate financial records. If violated, customers may see incorrect balances and the bank may face auditing problems.

7. Every Transaction Must Have a Unique ID

Each transaction should have a unique transaction identification number. This helps the bank identify and track individual transactions. Without it, duplicate transactions and difficulties in finding transaction records may occur.

Therefore, maintaining these invariants helps the banking application remain secure, reliable and consistent.

Q2. Entities and Value Objects (6 Marks)

In object-oriented domain modelling, the important concepts are classified into Entities and Value Objects.

An Entity is an object that has a unique identity and its properties can change during its lifetime.

A Value Object is an object that is identified by its values rather than a unique identity. It is generally immutable.

The classification of the given concepts is explained below.

1. Bank Account – Entity

A bank account is an entity because every account has a unique account number. Its balance and status can change through deposits, withdrawals and other operations. However, its identity remains the same throughout its lifetime.

2. Account Number – Value Object

An account number is a value object because it is identified by its actual value. It is used to identify a bank account and should not change after the account is created. Two account numbers are considered equal if their values are the same.

3. Money – Value Object

Money is a value object because it contains an amount and currency. For example, ₹5,000 INR represents a particular monetary value. It should be immutable so that its value cannot be changed unexpectedly during a transaction.

4. Ledger Entry – Entity

A ledger entry is an entity because every transaction record has a unique transaction ID. It contains details such as transaction amount, date, time and transaction type. Once a

transaction is completed, its record should remain permanent for future verification and auditing.

5. Overdraft Limit – Value Object

An overdraft limit is a value object because it represents a monetary amount that defines how much extra money a customer is allowed to withdraw. Its identity is based on its value and currency, not on a separate unique identity.

6. Fund Transfer – Entity

A fund transfer is an entity because every transfer has a unique transfer ID. It also has a lifecycle, such as initiated, pending, completed or failed. Its status changes as the transfer is processed.

Classification Summary

Concept	**Classification**	**Reason**
Bank Account	Entity	Has unique identity
Account Number	Value Object	Identified by value
Money	Value Object	Amount and currency
Ledger Entry	Entity	Has unique transaction ID
Overdraft Limit	Value Object	Identified by monetary value
Fund Transfer	Entity	Has unique ID and lifecycle

Thus, entities maintain identity and lifecycle, whereas value objects represent values and should generally remain immutable.

Q3. Self-Enforcing Objects and Java Design (8 Marks)

A self-enforcing object is an object that maintains its own data and ensures that business rules are followed whenever an operation is performed.

As explained in our notes, an object should not simply store data. It should also protect its own state and prevent invalid operations.

For the banking application, we can design a Bank Account class that contains account number, account holder name, balance, overdraft limit and account status.

All these important data members should be private so that external classes cannot directly modify them.

1. Deposit Money

The deposit operation should first check whether the transaction amount is greater than zero. It should also check that the account is not closed.

If all conditions are satisfied, the amount is added to the account balance and a corresponding ledger entry is created.

For example, if the current balance is ₹10,000 and the customer deposits ₹2,000, the new balance becomes ₹12,000.

2. Withdraw Money

Before allowing a withdrawal, the system should check the transaction amount, account status and overdraft limit.

If the account is frozen or closed, the withdrawal should be rejected. The withdrawal should also be rejected if it exceeds the permitted overdraft limit.

For example:

- Current balance = ₹10,000
- Overdraft limit = ₹5,000
- Maximum permitted withdrawal = ₹15,000

If the customer withdraws ₹12,000, the remaining balance becomes -₹2,000. This is allowed because it is within the overdraft limit.

However, if the customer tries to withdraw ₹16,000, the transaction should be rejected.

3. Freeze an Account

The freeze operation changes the account status from ACTIVE to FROZEN.

After freezing, the customer should not be allowed to withdraw money or transfer funds from that account. Only an authorized person should be able to freeze the account.

4. Unfreeze an Account

The unfreeze operation changes the account status from FROZEN to ACTIVE.

Only an authorized person should be allowed to perform this operation. After unfreezing, the customer can continue normal banking transactions.

5. Close an Account

Before closing an account, the bank should check whether there are any pending transactions or outstanding amounts.

After completing the necessary checks, the account status is changed to CLOSED. Once the account is closed, further transactions should not be permitted.

How Does the Design Protect the Account?

1. Invalid Transaction Amounts: The system validates every transaction amount and rejects zero or negative values.

2. Account Status: Before processing withdrawals, the system checks whether the account is active, frozen or closed.

3. Overdraft Limit: The system checks the available balance and overdraft limit before allowing a withdrawal.

4. Encapsulation: Account details are declared private, and external classes can access or modify them only through controlled methods.

5. Validation: Every operation checks the required conditions before changing the account state.

This follows the same concept of encapsulation and validation explained in our notes using the ParkingSpot example.

Therefore, self-enforcing objects prevent invalid states and make the banking application more secure and reliable.

Q4. Money Transfer and Aggregate Consistency (6 Marks)

Consider a customer transferring ₹5,000 from Account A to Account B.

During the transfer, the system successfully deducts ₹5,000 from Account A but crashes before crediting Account B.

1. Problems That Can Arise

The main problem is that money is deducted from the sender's account but is not credited to the receiver's account.

This can result in:

- Loss of money for the customer.
- Incorrect account balances.
- Incomplete transaction history.
- Customer complaints and disputes.
- Financial and accounting problems for the bank.

2. Business Invariants Violated

The following invariants are violated:

- Money transfer must be completed successfully.

- Account balance must match the transaction history.
- Every transfer must maintain consistency between the sender and receiver accounts.
- Money should not be lost during a transaction.

3. How to Prevent Money Loss

To prevent this problem, the bank should use database transactions and follow ACID properties.

ACID stands for:

- Atomicity: Both debit and credit operations should succeed together. If one operation fails, the entire transaction should be rolled back.
- Consistency: All banking business rules must remain valid before and after the transaction.
- Isolation: Multiple transactions should not interfere with one another.
- Durability: Once a transaction is successfully committed, its changes must be permanently stored.

For example, when transferring ₹5,000:

1. First, the system checks whether both accounts are valid.
2. It checks the sender's available balance and overdraft limit.
3. It deducts ₹5,000 from Account A.
4. It credits ₹5,000 to Account B.
5. It records the transaction in both account ledgers.
6. Finally, the transaction is committed.

If any operation fails before the transaction is committed, the system should roll back the changes.

4. Handling Failures During Transfer

The system should use rollback and recovery mechanisms.

If the system crashes before the transaction is committed, the database should recover to the previous consistent state.

If the transfer is already committed, the transaction should remain completed even if the application crashes afterward.

The bank should also maintain a unique transaction ID to identify the transfer and prevent the same transaction from being processed twice.

5. Transfers Between Different Banking Systems

If the two accounts belong to different banks, using a single database transaction may not be possible.

In this situation, the bank can use a distributed transaction mechanism or the Saga pattern.

The sending bank first processes the debit request. The receiving bank then processes the credit request. If the credit operation fails, a compensating transaction is initiated to refund the sender.

Both banks should maintain transaction records and update the transfer status.

Thus, proper transaction management and failure handling help prevent money loss and maintain consistency.

Q5. Ledger Management and Design Evaluation (4 Marks)

A ledger is a record of all financial transactions performed in a bank account. It helps the bank maintain transaction history and verify account balances.

1. Information a Ledger Entry Should Contain

Each ledger entry should contain the following details:

- Unique Transaction ID.
- Account Number.
- Transaction Type (Deposit, Withdrawal or Transfer).
- Transaction Amount.
- Date and Time.
- Debit or Credit Details.
- Previous Account Balance.
- Updated Account Balance.
- Transaction Description.
- Transaction Status.
- Transfer Reference Number, if applicable.

These details help the bank identify and verify every transaction.

2. Should Existing Ledger Entries Be Modified or Deleted?

No, completed ledger entries should not be modified or deleted.

A ledger entry is a permanent financial record. It is required for auditing, verification and resolving customer complaints.

If a transaction is incorrect, the bank should create a new reversal or correction entry instead of deleting the original transaction.

This maintains transparency and prevents fraudulent changes to transaction history.

3. How to Maintain Balance Consistency

The bank should record every deposit, withdrawal and transfer in the ledger.

The account balance and ledger entries should be updated within the same database transaction. The bank should also perform regular reconciliation to verify that the account balance matches the transaction history.

The balance can be calculated using the following formula:

Current Balance = Opening Balance + Total Credits − Total Debits

This helps maintain accurate account balances.

4. Challenges When Transaction History Becomes Very Large

When the number of transactions increases, the bank may face problems such as:

- Storage: A large amount of storage is required to maintain transaction records.
- Performance: Searching and retrieving old transactions may become slower.
- Backup: Maintaining backups of large transaction histories becomes difficult.
- Reporting: Generating account statements and audit reports may take more time.
- Scalability: The system must handle an increasing number of customers and transactions.

To solve these problems, the bank can use database indexing, partitioning, archiving and regular backups.

Thus, proper ledger management helps maintain accurate financial records and improves the reliability of the banking application.